Webex Platform Playbook

How the builder agent interviews you, designs a deployment plan, loads domain skills, executes via wxcli, and verifies the results — plus a CUCM migration pipeline with assessment reports

PIPELINE

Setup → interview → design → execute → verify

EXECUTION LAYER

wxcli CLI → raw HTTP fallback

DOMAIN SKILLS

24 skills loaded on demand per task domain

COVERAGE

179 command groups · 10 OpenAPI specs · 51 reference docs

Overview Setup Interview Design Execute Verify Standalone Resilience

OVERVIEW

Webex Platform Playbook

Auto-selected on load

You describe what you want to build across any Webex API surface — calling, admin, devices, or messaging — and the playbook interviews you, designs a deployment plan, loads the right domain skills, executes via CLI, and verifies every resource it creates.

The problem

The Webex platform spans thousands of API endpoints across calling, admin/identity, device management, and messaging. Building anything non-trivial means coordinating resources in the right dependency order across multiple API domains, with the right IDs, scopes, and token types. Doing this by hand through Control Hub or raw API calls is slow and error-prone.

What the playbook does

The **wxc-calling-builder** agent (Sonnet) runs as a subagent with its own context window. It walks through five phases: setup your CLI environment, interview you about what to build, design a deployment plan with real resource IDs, load the relevant domain skill(s) for gotchas and CLI command mappings, execute the plan via `wxcli` commands, and verify every resource it created. For CUCM migrations, the **migration-advisor** agent (Opus) provides CCIE-level architectural reasoning, decision review, and dissent analysis grounded in an 8-doc knowledge base.

Platform at a glance

CLI GROUPS	DOMAIN SKILLS	OPENAPI SPECS	REFERENCE DOCS
179 command groups	24 specialized runbooks	10 (calling, admin, device, messaging, meetings, CC, wholesale, broadworks, UCM, TTS)	51 in docs/reference/

EXECUTION

`wxcli → raw HTTP`

ARCHITECTURE

`CLAUDE.md + 2 agents + 24 skills`

API domains covered

DOMAIN	SPEC	WHAT IT COVERS
Calling	webex-cloud-calling.json	Provisioning, features, settings, routing, call control, reporting
Admin	webex-admin.json	Org management, identity/SCIM, licensing, audit, hybrid, partner ops
Devices	webex-device.json	Phone/DECT provisioning, RoomOS configs, xAPI, workspaces
Messaging	webex-messaging.json	Spaces, teams, memberships, messages, bots, adaptive cards, ECM
Meetings	webex-meetings.json	Meeting CRUD, transcripts, recordings, Video Mesh, polls, Q&A
Contact Center	webex-contact-center.json	CC agents, queues, entry points, flows, campaigns, analytics (49 groups)
Wholesale	webex-wholesale.json	Wholesale partner provisioning APIs
BroadWorks	webex-broadworks.json	BroadWorks provisioning and subscriber management
UCM	webex-ucm.json	UCM Cloud connected provisioning APIs
TTS	tts-spec.json	Text-to-speech announcement generation

OVERVIEW

Project Structure

Reference — where things live

Ten zones make up the project — five you interact with directly (CLI, agents, skills, specs, knowledge base) and five that work behind the scenes (generator, reference docs, migration engine, prompts, tests).

How the pieces connect

Top row: OpenAPI specs are parsed by the generator into CLI commands. The agents execute those commands, guided by skills that draw on reference docs. Bottom row: the CUCM migration pipeline flows from discovery through assessment report generation.

Folder tree

```
webexCalling/
├── .claude/  AGENTS & SKILLS
│   ├── agents/
│   │   ├── wxc-calling-builder.md — main agent (Sonnet): setup → interview → design → execute → verify
│   │   └── migration-advisor.md — CCIE-level migration advisor (Opus): architectural reasoning + decision review
│   ├── skills/ — 24 domain skills loaded on demand
│   │   ├── provision-calling/ — users, locations, licenses
│   │   ├── configure-features/ — AA, CQ, HG, paging, park, pickup, VM groups
│   │   ├── customer-assist/ — CX Essentials (screen pop, wrap-up, recording, supervisors)
│   │   ├── manage-call-settings/ — 39+ person/workspace settings
│   │   ├── configure-routing/ — trunks, dial plans, PSTN
│   │   ├── manage-devices/ — phones, DECT, workspaces
```

```
│   ├── device-platform/ — RoomOS configs, xAPI, 9800-series
│   ├── call-control/ — real-time call ops, webhooks, XSI
│   ├── reporting/ — CDR, queue stats, call quality
│   ├── reporting-cc/ — Contact Center analytics (queue/agent stats, EWT)
│   ├── reporting-meetings/ — meetings quality, workspace metrics, live monitoring
│   ├── manage-identity/ — SCIM, directory, groups
│   ├── manage-licensing/ — audit, assign, reclaim
│   ├── audit-compliance/ — admin/security events
│   ├── messaging-spaces/ — spaces, teams, ECM
│   ├── messaging-bots/ — bots, adaptive cards, webhooks
│   ├── manage-meetings/ — schedule, manage, query meetings + content
│   ├── org-health/ — 18-check org health assessment + HTML report
│   ├── query-live/ — natural-language read-only queries against live org state
│   ├── contact-center/ — CC agents, queues, flows, campaigns
│   ├── video-mesh/ — Video Mesh clusters, nodes, thresholds
│   ├── teardown/ — dependency-safe cleanup (13-layer deletion)
│   ├── cucm-migrate/ — CUCM-to-Webex execution (DB-driven)
│   └── wxc-calling-debug/ — systematic error diagnosis
└── settings.json — shared permissions (pre-approves wxcli)
│
├── src/wxcli/  CLI SOURCE
│   ├── main.py — entry point, 179 command groups + whoami/switch-org/clear-org
│   ├── config.py — token storage, orgId, CC region, profile management
│   ├── auth.py — API client init
│   ├── output.py — table/JSON output formatting
│   ├── errors.py — error detection + user-facing tips
│   ├── commands/ — 172 generated modules (registered via _registry.py) + 5-6 hand-
written exception files
│   │   ├── _registry.py — generator-emitted registration manifest (GENERATED_GROUPS);
main.py loops over it instead of hand-maintained registrations
│   │   ├── cleanup.py — batch cleanup (13-layer dependency-safe deletion)
│   │   ├── cucm.py — CUCM migration pipeline CLI commands
│   │   ├── converged_recordings_export.py — hand-written: download/export recordings
│   │   └── cc_*.py — 50+ contact center command modules
│   ├── org_health/  ORG HEALTH ASSESSMENT (18 CHECKS)
│   │   ├── models.py — Finding, CategoryScore, HealthResult dataclasses
│   │   ├── collector.py — load manifest, validate collected JSON
│   │   ├── checks.py — 18 check functions + @check() decorator registry
│   │   ├── analyze.py — load → check → write results.json
```

```
│   │   └── report.py — read results → generate self-contained HTML report
│   └── migration/  MIGRATION ENGINE (11 PHASES)
│       ├── models.py — 26 data types, 21 DecisionTypes
│       ├── store.py — SQLite store: objects, cross-refs, decisions, journal
│       ├── cucm/ — AXL connection, 16 extractors, Unity voicemail, discovery
│       ├── transform/ — 27 normalizers, 19 mappers, 13 analyzers + advisor
│       ├── execute/ — planner, NetworkX DAG, batch execution
│       ├── advisory/ — 20 recommendation rules + 20 cross-cutting patterns
│       ├── report/ — HTML/PDF assessment report ("Authority Minimal" design)
│       ├── export/ — deployment plan, JSON/CSV exports
│       └── preflight/ — 8 pre-execution checks
│
├── tools/  GENERATOR PIPELINE
│   ├── generate_commands.py — orchestrator: parse spec → render → write
│   ├── spec_sync.py — atomic entrypoint: pull spec updates → regen all tracked specs → update _registry.py → run drift gate
│   ├── drift_check.py — coherence gate: spec↔CLI parity, dead wxcli refs, published-count & unreferenced-group checks
│   ├── openapi_parser.py — OpenAPI 3.0 → Endpoint objects
│   ├── command_renderer.py — Endpoint objects → Click command files
│   ├── field_overrides.yaml — table columns, display config, bug fixes
│   ├── postman_spec_diff.py — offline Postman ↔ spec gap diff
│   └── create_stress_test_db.py — stress test DB creation (complex CUCM environments)
│
├── specs/  OPENAPI SPECS (10)
│   ├── webex-cloud-calling.json — calling APIs
│   ├── webex-admin.json — admin/org management APIs
│   ├── webex-device.json — device management APIs
│   ├── webex-messaging.json — messaging APIs
│   ├── webex-meetings.json — meetings/video mesh APIs
│   ├── webex-contact-center.json — contact center APIs (49 groups, 417 commands)
│   ├── webex-wholesale.json — wholesale partner APIs
│   ├── webex-broadworks.json — BroadWorks provisioning APIs
│   ├── webex-ucm.json — UCM Cloud connected APIs
│   └── tts-spec.json — text-to-speech announcement APIs
│
├── docs/
│   ├── reference/  51 API REFERENCE DOCS
│   │   ├── authentication.md — token types, scopes, OAuth flows
```

```
│   │   ├── provisioning.md — people, licenses, locations
│   │   ├── call-features-*.md — AA, CQ, HG, paging, park, CX Essentials (2)
│   │   ├── person-call-settings-*.md — handling, media, permissions, behavior (4)
│   │   ├── self-service-call-settings.md — 151 /people/me/ endpoints
│   │   ├── location-call-settings-*.md — core, media, advanced (3)
│   │   ├── devices-*.md — core, DECT, workspaces, platform/RoomOS (4)
│   │   ├── call-routing.md — trunks, route groups, dial plans, PSTN
│   │   ├── messaging-*.md — spaces, bots (2)
│   │   ├── meetings-*.md — core, content, settings, infrastructure (4)
│   │   ├── contact-center-*.md — core, routing, analytics, journey (4)
│   │   ├── admin-*.md — org, identity, licensing, audit, hybrid, partner, apps (7)
│   │   └── wxcadm-*.md — wxcadm library: XSI, E911, CP-API (8)
│   ├── knowledge-base/migration/   8 MIGRATION KB DOCS
│   │   ├── kb-css-routing.md — CSS/partition → calling permissions
│   │   ├── kb-device-migration.md — device replacement paths, firmware conversion
│   │   ├── kb-trunk-pstn.md — trunk topology, LGW vs CCPP
│   │   ├── kb-feature-mapping.md — HG → CQ depth, AA mapping, shared lines
│   │   ├── kb-user-settings.md — call forwarding, voicemail, permissions
│   │   ├── kb-location-design.md — device pool → location consolidation
│   │   ├── kb-identity-numbering.md — DN ownership, extension conflicts
│   │   └── kb-webex-limits.md — platform hard limits, feature gaps (always loaded)
│   ├── plans/ — 55+ deployment plans + execution reports
│   ├── prompts/ — 85+ build & phase prompts driving Claude Code sessions
│   ├── templates/ — deployment plan + execution report templates
│   ├── team-prompts/ — agent team patterns (spec-to-ship, reference-audit)
│   ├── arch/ — 2026-07 coherence audit trail: inventory, drift findings, target design,
refactor plan (dated snapshot, not living docs)
│   └── wxc-pipeline-visual.html — this page
│
├── tests/   3325 TESTS
│   ├── migration/ — normalizers, mappers, analyzers, advisory, preflight, report
│   ├── test_partner_e2e.py — partner multi-org end-to-end tests
│   └── test_org_id_injection.py — verifies 1150/1839 commands inject orgId
│
└── CLAUDE.md — master project context (agent reads this first)
```

Key rule: CLI commands in `src/wxcli/commands/` are generated, never hand-edited. Fix bugs in `tools/field_overrides.yaml` and regenerate. The old "legacy trio" is fully retired: `locations.py` and `numbers.py` were generator output all along, and `licenses.py` was consolidated into the generated `licenses` group on 2026-07-02 (`licenses-api` survives only as a deprecated one-release alias). The remaining hand-written files — `update.py` , `configure.py` , `cucm.py` + `cucm_config.py` , `cleanup.py` , `converged_recordings_export.py` — are deliberate exceptions per the generator's charter, not legacy debt. Registration is now generator-owned via `_registry.py` (ADR-9), so a generated-but-unregistered module can no longer happen silently.

SETUP

Check CLI Environment

Runs silently on first invocation

Before anything can happen, the agent confirms wxcli is installed and working — so you never get halfway through a build only to discover the tool is missing.

What happens

The agent runs `wxcli --help` to verify the CLI is installed and on the PATH. If it is not found, the agent installs it from the local source tree with `pip install -e .` from the repo root.
The agent also confirms Python is available (`python3.14`) since some operations — org health analysis, migration pipeline — run as Python modules rather than CLI commands.

What wxcli covers

COMMAND GROUPS	OPENAPI SPECS	GENERATOR	FALLBACK
179	10	OpenAPI 3.0 → Click CLI	raw HTTP

SETUP

Validate Authentication Token

Every session, before any API call

A stale or missing token is the #1 reason builds fail — so the agent checks it first and fixes it interactively if needed.

What happens

The agent runs `wxcli whoami`. If the token is valid, you see your name, email, and org. If it fails, the agent asks for a new token and pipes it into the CLI:

```
# This is the only way that works in Claude Code
# (each Bash call is a new shell, so export doesn't persist)
echo "<TOKEN>" | wxcli configure
```

This saves the token to `~/.wxcli/config.json` where it persists across shell invocations.

Token types

TYPE	LIFETIME	USE CASE
Personal	12 hours	Quick testing, one-off builds
OAuth	14 days (access) / 90 days (refresh)	Longer sessions, automated flows
Service App	Via refresh	Production automation
Bot	Never expires	Messaging bots, always-on integrations (limited scope)

Partner & multi-org tokens

If your token can see more than one organization (common for partners, VARs, and MSPs managing customer orgs), commands need to know **which org to target**. The CLI handles this automatically:

- **Auto-detection** — During `wxcli configure`, the CLI queries `/v1/organizations`. If more than one org comes back, it lists them and asks you to pick one. Your selection is saved to config alongside the token.
- **Transparent injection** — 1150 of 1839 CLI commands automatically inject the saved `orgId` into every API call. No `--org-id` flag needed — it happens silently.
- **Single-org tokens** — If your token sees only one org (the common case), none of this applies. The CLI skips org selection and proceeds normally.

Two identities in `whoami`

When an org is targeted, `wxcli whoami` shows two distinct lines:

```
# Your identity (from the token)
User:  Alice Partner (alice@partner.com)
Org:   Y2lz...abc  (Partner Inc)

# Where commands will execute (from saved config)
Target: Y2lz...xyz  (Acme Corp)

# Token lifetime
Expires: 11h 42m remaining
```

`Org:` is who you are. `Target:` is whose org your commands modify. If there is no `Target:` line, commands run against your own org.

Switching orgs at runtime

COMMAND	WHAT IT DOES
`wxcli switch-org`	Lists all accessible orgs, lets you pick a different target (or pass an org ID directly)
`wxcli clear-org`	Removes the saved target — reverts to single-org behavior
`wxcli whoami`	Shows current identity + target org (if set) + token expiration

Safety gate for partner tokens: When the builder agent detects a `Target:` line in `whoami` output, it hard-stops and asks you to confirm: "You are targeting [**org name**]. Is this correct?" No commands execute until you confirm. If it is wrong, the agent runs `wxcli switch-org` and re-confirms. This prevents accidentally provisioning resources to the wrong customer org.

Gotcha discovered in session: `export WEBEX_ACCESS_TOKEN=...` does not work in Claude Code because each Bash tool call runs in a fresh shell. The pipe-based `echo | wxcli configure` pattern was discovered through trial and error and is now the standard approach.

SETUP

Check for Existing Deployment Plans

After auth is confirmed

If a previous session already designed a plan, you can pick up where it left off instead of starting from scratch.

What happens

The agent reads `docs/plans/` looking for existing deployment plans. If it finds one, it asks: "Do you want to continue this work or start fresh?" This is especially important when the agent's context compacts mid-build — it can recover by reading the saved plan file.

For messaging requests, the agent also checks for messaging reference docs (`messaging-spaces.md` , `messaging-bots.md`) since those are newer additions to the platform.

Why plans are saved to disk

- **Context recovery** — The builder agent runs as a subagent. If its context compacts mid-build (88+ tool calls), it can re-read the plan and resume
- **Audit trail** — Every build gets a plan file + execution report, so you can see exactly what was done
- **Cross-session continuity** — A new session can pick up a partially-executed plan

INTERVIEW

Identify What You Want to Build

After setup completes

You describe the goal in your own words, and the agent maps it to the right domain, scope, and skill — so it loads exactly the right expertise for your task.

How the interview works

The agent asks questions **one at a time**. First: "What do you want to build or configure?" Then: "At what scope?" It listens for domain signals to determine which of the 24 skills to load during execution.

Domain routing (24 skills)

YOU SAY	SKILL LOADED
"Set up an auto attendant with a call queue"	CONFIGURE-FEATURES
"Configure screen pop and wrap-up reasons"	CUSTOMER-ASSIST
"Provision 20 users in Denver"	PROVISION-CALLING
"Enable call forwarding for the sales team"	MANAGE-CALL-SETTINGS
"Set up PSTN trunks and dial plans"	CONFIGURE-ROUTING
"Provision DECT base stations"	MANAGE-DEVICES
"Deploy RoomOS configs to conference rooms"	DEVICE-PLATFORM
"Set up call recording and webhooks"	CALL-CONTROL
"Pull last month's CDR and queue stats"	REPORTING
"Set up SCIM sync from Azure AD"	MANAGE-IDENTITY
"Audit admin activity for compliance"	AUDIT-COMPLIANCE
"Reclaim unused licenses"	MANAGE-LICENSING
"Create a team space and add members"	MESSAGING-SPACES
"Build a bot that sends adaptive cards"	MESSAGING-BOTS
"Schedule a Webex meeting with breakouts"	MANAGE-MEETINGS

"Set up CC agents and queue routing"	CONTACT-CENTER
"Check Video Mesh cluster health"	VIDEO-MESH
"Clean up the test location"	TEARDOWN
"Migrate our CUCM to Webex Calling"	CUCM-MIGRATE
"Pull CC queue stats and agent performance"	REPORTING-CC
"Check meeting quality and workspace metrics"	REPORTING-MEETINGS
"Run an org health audit"	ORG-HEALTH
"Show me all users in Denver with call forwarding"	QUERY-LIVE
"This API call is failing with 403"	WXC-CALLING-DEBUG

Scope varies by domain

- **Calling** — Org-wide, location(s), or user(s)
- **Messaging** — Space-scoped, team-scoped, bot-scoped, or org-wide audit
- **Meetings** — Per-meeting, site-wide, or infrastructure (Video Mesh)
- **Admin** — Org-level settings, SCIM directory, license inventory
- **Devices** — By location, by workspace, or individual device
- **Contact Center** — Per-queue, per-agent, or full CC environment

INTERVIEW

Check Prerequisites and Gather Constraints

After objective is clear

The agent does not just ask if prerequisites exist — it runs commands to check, then gathers your naming conventions, schedule patterns, and compliance requirements before writing the plan.

Live prerequisite checks

Based on the objective, the agent runs targeted queries against your live org:

```
# Do the target locations exist and have calling enabled?
wxcli locations list --calling-only

# Are there enough licenses?
wxcli licenses list

# Do the target users exist in the right location?
wxcli users list --location LOCATION_ID --output table

# Are any extensions already in use?
wxcli numbers list --location LOCATION_ID -o json
```

It reports what is confirmed, what is missing, and any conflicts (e.g., extension ranges already in use, duplicate schedule names, insufficient license counts).

Prerequisites vary by domain

Each API domain has a different prerequisite shape. The agent checks for the right things based on what you are building:

DOMAIN	WHAT GETS CHECKED
Calling	Calling-enabled locations, available licenses, user existence, extension conflicts, number inventory
Messaging	Token type (bot vs. user vs. admin), webhook callback URL readiness, space/team existence
Meetings	Webex site URL (multi-site orgs), personal room vs. scheduled, recurring series requirements
Contact Center	CC region (`wxcli set-cc-region`), CC-scoped OAuth (not PAT — PATs lack `cjp:config` scopes), inbound vs. outbound, routing type

Devices	Target workspace/user existence, activation code availability, 5-device-per-user limit, DECT base station capacity
Routing	Trunk registration status, existing dial plan conflicts, route group/list dependency chain, PSTN provider type

Constraints gathered

After confirming prerequisites, the agent asks about business rules that shape the plan:

- **Naming conventions** — Site prefixes, department suffixes, feature naming patterns (e.g., "All hunt groups prefixed with location code")
- **Extension ranges** — Which ranges are reserved for which feature types (e.g., "4-digit starting with 3xxx for Sales")
- **Schedules** — Business hours, holidays, per-day overrides (each day must be a separate event in the API)
- **Compliance** — Call recording requirements, E911 requirements, audit log retention
- **Integration** — Existing PBX migration constraints, PSTN provider limits, SBC configuration

Tool selection

The agent decides which execution layer(s) to use based on what the build requires:

TOOL	WHEN USED
`wxcli`	Primary — 179 command groups, covers all standard CRUD operations
Raw HTTP	AA menu key configurations, deeply nested settings, operations not yet in wxcli

Most builds never leave wxcli. Raw HTTP is used for a handful of operations where the CLI generator skips deeply nested fields (e.g., AA menu `keyConfigurations`).

DESIGN

Build the Deployment Plan

After interview completes

The agent writes a complete, executable plan with real resource IDs, actual CLI commands, and rollback steps — so you can review the entire build before a single API call is made.

Reference doc loading

Based on the domain identified in step 4, the agent loads the relevant reference docs from `docs/reference/` (51 docs covering all ten API surfaces). These contain SDK method signatures, raw HTTP endpoints, parameter requirements, and gotchas discovered through live testing. For complex builds spanning multiple domains, the agent loads docs from each domain — a multi-tier call flow might load provisioning, call features, and routing docs together.

Plan structure

Every plan follows the template at `docs/templates/deployment-plan.md` and includes seven sections:

- **Objective** — One paragraph: what you are building
- **Prerequisites** — Table with verification method and status (CONFIRMED / TO VERIFY / NEEDS CREATION)
- **Key IDs** — Real resource IDs from the live org, confirmed via API — never placeholders
- **Execution steps** — Numbered, dependency-ordered, each with the actual `wxcli` command, expected result, and "depends on" reference
- **Rollback plan** — If step N fails, exactly what to undo from steps 1..N-1
- **Verification steps** — How to confirm each resource after creation
- **Estimated execution** — Sync vs. async, approximate time for bulk operations

Dependency ordering

The plan enforces creation order based on resource dependencies. The agent traces the dependency chain for each resource type:

MUST EXIST FIRST	BEFORE CREATING
Location (calling-enabled)	Any location-scoped resource
Schedules (business hours, holidays)	Auto attendants, call queues
Users (with calling license)	Hunt group/queue agents, AA menu transfers

Auto attendants, hunt groups, call queues	AA menu key configurations (transfer targets)
Trunks	Route groups
Route groups	Route lists
Route lists	Dial plans

Deletion order is the exact reverse — dial plans first, locations last. The `wxcli cleanup` command enforces this as a 13-layer cascade.

Example plan excerpt

```
### Step 2a — Create "Business Hours" schedule
wxcli location-call-settings create-schedules \
  Y2lzY29z... \
  --name "Business Hours" --type businessHours
# Depends on: Step 1 (location confirmed)
# Expected: schedule_id returned

### Step 3 — Create "Main Menu" auto attendant
wxcli auto-attendant create Y2lzY29z... \
  --name "Main Menu" --extension 1000 \
  --business-hours-schedule-id <FROM_STEP_2a>
# Depends on: Step 2a (schedule), Step 2b (holiday schedule)
# Expected: aa_id returned
```

Real plans use actual base64 IDs from the live org, not placeholders. The only deferred value is an ID from a prior creation step (marked `<FROM_STEP_N>`).

Saved to disk

The plan is saved to `docs/plans/YYYY-MM-DD-{name}.md` before execution starts. This serves three purposes:

- **Compaction recovery** — The builder agent runs as a subagent. If its context compacts mid-build (88+ tool calls), it re-reads the plan file and resumes from the next pending step
- **Audit trail** — Every build gets a plan file paired with an execution report, so you can see exactly what was intended and what actually happened
- **Cross-session continuity** — A new session can pick up a partially-executed plan without re-running the interview

DESIGN

CUCM Migration Advisory System

For CUCM migrations — after pipeline analysis

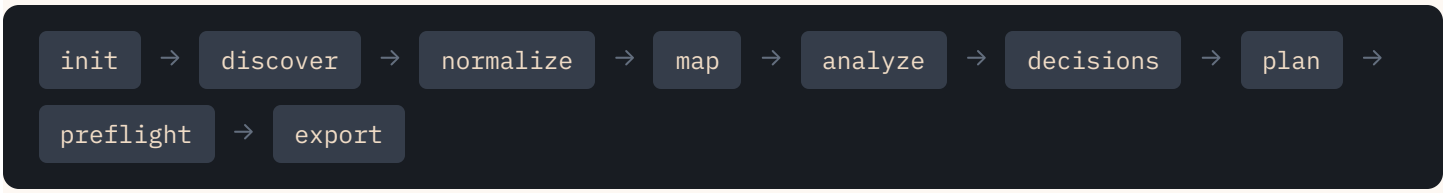
For CUCM migrations, an Opus-powered CCIE-level advisor reads every decision the pipeline made, cross-references against an 8-doc knowledge base, flags where heuristics are wrong, and produces a professional assessment report — so the migration is grounded in architectural reasoning, not just pattern matching.

Two agents, two roles

AGENT	MODEL	ROLE
wxc-calling-builder	Sonnet	Runs the pipeline, executes commands, orchestrates the workflow
migration-advisor	Opus	Reads all decisions, applies architectural reasoning, produces narratives, flags dissents

The builder runs the 11-phase pipeline mechanically. The advisor is invoked twice: once to produce a **migration narrative** (cross-decision analysis + risk assessment), and again during **decision review** to present advisories and recommendations with contextual explanation.

The full migration pipeline



All 11 phases run via `wxcli cucm` commands against a SQLite store. The advisory system activates after the `analyze` phase. Assessment reports can be generated at any point after `analyze` without running the execution phases.

Layer 1: Per-decision recommendations

20 recommendation rules in `recommendation_rules.py` — one function per DecisionType. Each reads the decision's context and options, returns a recommended choice + reasoning, or `None` for genuinely ambiguous cases. Honest uncertainty is a feature, not a bug.

DECISION TYPE	EXAMPLE RULE
FEATURE_APPROXIMATION	Queue features enabled → Call Queue. >8 agents → Call Queue. ≤4 top-down → Hunt Group. 5-8 → ambiguous

DEVICE_INCOMPATIBLE	Model lookup: 7811→9841, 7832→room device, ATA 190→ATA 192. Includes firmware type reasoning
CSS_ROUTING_MISMATCH	Partition ordering dependency → "manual". Scope difference → "use_union" unless conflicts
MISSING_DATA	Almost always "skip" with remediation guidance. Trunk password → "generate"

Layer 2: Cross-cutting advisory patterns

20 pattern detectors in `advisory_patterns.py` — each reads the full canonical model plus all prior decisions and finds patterns that span multiple objects. These are the insights a CCIE would spot that no single-decision rule can catch.

CATEGORY	PATTERNS
CRITICAL	Partition ordering loss (Webex uses longest-match, no ordering), CPN transformation chains (5-level → flat caller ID), PSTN connection type classification
ELIMINATE	Restriction CSS → calling permissions, translation pattern elimination, partition-based time routing → AA schedules, over-engineered dial plan removal, voicemail pilot simplification
REBUILD	Hunt pilot reclassification (queue behavior → Call Queues), location consolidation, shared line → virtual extensions, trunk destination consolidation
INFO	Device bulk upgrade planning, dial plan style detection, media resource removal, E911/CER migration flag
TIER 4	Recording-enabled users, SNR configured users, transformation patterns, extension mobility usage

Knowledge base

The advisor reads from **8 curated KB documents** in `docs/knowledge-base/migration/` , selected dynamically based on which DecisionTypes and advisory patterns are present. `kb-webex-limits.md` (platform hard limits and feature gaps) is always loaded.

CSS & ROUTING `kb-css-routing.md`	DEVICE MIGRATION `kb-device-migration.md`	TRUNK & PSTN `kb-trunk-pstn.md`	FEATURE MAPPING `kb-feature-mapping.md`
USER SETTINGS `kb-user-settings.md`	LOCATION DESIGN `kb-location-design.md`	IDENTITY & NUMBERING `kb-identity-numbering.md`	PLATFORM LIMITS `kb-webex-limits.md`

Dissent analysis

The advisor can **disagree with the static heuristics**. When the rules say one thing but the architectural context suggests another, the advisor flags a structured dissent — citing which KB section informs its reasoning, which heuristic it disagrees with, and why. Dissents are presented to the admin during decision review so they can make informed choices, not blind ones.
Grounding priority: **(1)** knowledge base docs, **(2)** static heuristic output, **(3)** the model's own training. If the advisor can't answer from any of these three sources, it says so explicitly rather than speculating.

Assessment report

After the analysis phase, `wxcli cucm report` generates a professional **HTML/PDF assessment report** designed for Sales Engineers to hand to customers showing that CUCM-to-Webex migration isn't as hard as they think.

DESIGN SYSTEM	EXECUTIVE SUMMARY	TECH APPENDIX	COMPLEXITY SCORE
"Authority Minimal" v4	4 pages	22 sections (A–V)	0–100 (7 weighted factors)

CHARTS	TYPOGRAPHY
SVG gauge, donut, stacked bars	Lora + Source Sans 3 + IBM Plex

The report reads directly from the SQLite store — no plan, preflight, or export required. It includes a complexity gauge with tier-based coloring (green/amber/red), environment inventory with donut charts, effort bands (auto/planning/manual), and a full technical appendix with every decision explained in customer-friendly language.

Strategic purpose

Prevent customers from migrating to Microsoft Teams by demystifying the CUCM-to-Webex migration path with data-backed evidence. The report transforms the opaque "how hard is this migration?" question into a scored, explained, actionable document — with every decision grounded in real CUCM inventory data.

> **Key insight:** The static heuristics (rules + patterns) are the backbone — tested, deterministic, and fast. The Opus advisor layer adds interpretation and structured dissent for cases where mechanical rules are likely wrong. Falls back to mechanical presentation if the advisor agent is unavailable.

DESIGN

Approval Gate: Review Before Execution

Plan presented, awaiting your go/no-go

You see every command that will run against your org before it runs — so there are no surprises during execution.

How it works

The agent presents the full deployment plan and waits. It will not execute a single command until you explicitly approve. If something looks wrong, you say so, the agent updates the plan, and re-presents it. Even "just do it" gets a plan shown first — this is non-negotiable per the agent's critical rules.

APPROVAL GATE

This is the only hard gate in the pipeline. Everything before it is reversible (just conversation). Everything after it modifies your Webex org. The one exception: single read-only operations (GET requests) used for discovery during the interview phase.

What to look for

- **Resource names** — Do they match your naming conventions?
- **Extensions/numbers** — Are they in the right range? Any conflicts?
- **Dependency order** — Schedules before AAs, AAs before menus, users before queue agents
- **Token scopes** — Does your token have the required scopes for each step?
- **Rollback plan** — Does it cover what you would need to undo?

EXECUTE

Load Domain Skills as Reference

Before running each domain's commands

Before running any domain's commands, the agent reads that domain's skill file — so it has the CLI command mappings, prerequisites, and hard-won gotchas that prevent trial-and-error failures.

How skill loading works

Skills are **not dispatched as subagents**. The agent reads each skill file inline as it enters that domain's steps in the deployment plan. The skill tells the agent which CLI commands to use, what can go wrong, and what raw HTTP fallbacks exist. Most builds touch multiple skills:

```
# Steps creating locations/users
→ read provision-calling/SKILL.md

# Steps creating AA, CQ, HG
→ read configure-features/SKILL.md

# Steps configuring person settings
→ read manage-call-settings/SKILL.md

# Steps setting up CC queues and agents
→ read contact-center/SKILL.md

# Steps scheduling meetings
→ read manage-meetings/SKILL.md

# Steps pulling CC analytics
→ read reporting-cc/SKILL.md

# Steps running org health audit
→ read org-health/SKILL.md

# On any error
→ read wxc-calling-debug/SKILL.md
```

The full list of 24 skills and their domains is in step 4's routing table. Each skill can also be invoked directly as a slash command (`/configure-features` , `/manage-call-settings` , etc.) for focused work outside the builder workflow.

Admin ops without skills

For admin operations without a dedicated skill (org settings, hybrid monitoring, partner ops), the agent handles them inline using reference docs. The threshold: handle inline when it is a single read-only command or the group has fewer than 5 commands. For destructive operations, always load the relevant reference doc first.

EXECUTE

Run Commands in Dependency Order

Plan steps executed one by one

Each command runs in the order the plan specified, with the output captured and reported in real time — so you see exactly what was created and can track progress.

Execution pattern

Commands run sequentially, never in parallel. Each must complete before the next starts. The agent reports progress in a consistent format:

```
# Step 1/6: Create business hours schedule
$ wxcli location-schedules create Y2lz... --name "Business Hours" --type
businessHours
Created: schedule_id=MDNi...

# Step 2/6: Create auto attendant "Main Menu"
$ wxcli auto-attendant create Y2lz... --name "Main Menu" --extension 8000
Created: aa_id=NWEy...

# Step 3/6: Configure AA menu (raw HTTP fallback)
$ python3 -c "..."
Updated: keyConfigurations for keys 1, 2, 0
```

Bulk operations

SCALE	METHOD
< 20 items	Shell loop with `wxcli` + `sleep 1` between calls
20-50 items	Shell loop with rate limiting
50+ items	Async Python SDK via `docs/reference/wxc-sdk-patterns.md`

> **Real-world gotcha:** AA menu configuration requires raw HTTP, not wxcli. The `keyConfigurations.value` field is mandatory even for non-transfer actions. And there is no `holidayMenu` field in the API — holiday routing uses the after-hours menu with a holiday schedule.

EXECUTE

Error Handling: Stop, Diagnose, Decide

When any command returns an error

The agent never silently continues past a failure — it stops, tells you what went wrong, and gives you three choices.

The error protocol

- **Stop immediately** — Do not continue to the next step
- **Show the full error** — HTTP status code + error message from wxcli
- **Diagnose** — Match the error against known patterns (401 = token, 403 = scope, 409 = conflict, 400 = bad params)
- **Suggest a fix** — Specific resolution based on the error type
- **Ask you** — (A) fix this and retry, (B) skip this step and continue, (C) rollback everything done so far

Common error patterns

ERROR	CAUSE	FIX
401 Unauthorized	Token expired	`echo "<TOKEN>" | wxcli configure`, resume
403 Forbidden	Missing scope	Get new token with required scopes
404 Not Found	Wrong resource ID	List resources, verify ID
409 Conflict	Duplicate name/extension	Use different name or extension
400 Bad Request	Invalid parameters	Run with `--debug`, read error body
429 Rate Limited	Too many requests	wxcli auto-retries; add `sleep 1` in loops

Debug skill escalation

For complex errors that do not match simple patterns, the agent can invoke `/wxc-calling-debug`. The debug skill runs targeted CLI commands with `--debug` flags, extracts tracking IDs for TAC escalation, and works through a systematic diagnosis flow.

VERIFY

Read Back Every Resource and Compare to Plan

After all execution steps complete

Every resource the agent created gets read back from the API and compared to what the plan specified — so you know the build matches your intent, not just that API calls returned 200.

What happens

For each resource in the deployment plan, the agent runs the corresponding `show` command with `--output json` and compares the returned values against what the plan specified. Discrepancies are flagged immediately.

```
# Verify each created resource
wxcli auto-attendants show LOCATION_ID AA_ID --output json
wxcli call-queues show LOCATION_ID QUEUE_ID --output json
wxcli hunt-groups show LOCATION_ID HG_ID --output json
wxcli users show PERSON_ID --output json

# Check: name, extension, agents, schedule, phone number
```

What gets compared

- **Names** — Exact match to plan
- **Extensions** — Correct value, no conflicts
- **Agent/member lists** — Right people assigned
- **Schedule associations** — Business hours and holiday schedules wired correctly
- **Phone numbers** — Assigned if plan specified one
- **Settings values** — Forwarding destinations, DND state, voicemail rings, etc.

VERIFY

Generate and Save the Execution Report

Final step of every build

The execution report captures everything that happened — successes, failures, resource IDs, and next steps — so the build is fully documented for future reference.

Report structure

- **Summary** — What was built, count of resources created
- **Step-by-step results** — Each step: done/failed/skipped, resource IDs, timestamps
- **Discrepancies** — Anything that did not match the plan
- **Failed steps** — Full error details + suggested remediation
- **Verification results** — Read-back confirmation per resource
- **Next steps** — Follow-up work (e.g., "upload custom greeting via Control Hub")

Where it lives

```
docs/plans/2026-03-18-multi-tier-call-flow.md        # Plan
docs/plans/2026-03-18-multi-tier-call-flow-report.md # Report
```

Plan and report are always saved side-by-side. A future session can read both to understand what was built and what state it is in.

Gotcha sync protocol

If the build discovered new gotchas (API behaviors that differ from documentation), those get written back into the relevant reference doc in `docs/reference/`. This is how the playbook gets smarter over time — every session that discovers something new updates the docs so the next session does not hit the same issue.

STANDALONE

Org Health Assessment

On demand — "audit my org" or /org-health

A deterministic, three-phase org audit that collects live state via wxcli, runs 18 Python checks across four categories, and generates a self-contained HTML report — no LLM analysis, just code checking data.

Three-phase pipeline

```
Phase 1: Collect → Phase 2: Analyze → Phase 3: Report
```

Each phase is a discrete step. Phase 1 runs 14+ wxcli commands and saves JSON. Phase 2 runs deterministic Python checks against the collected data. Phase 3 generates a branded HTML report.

Phase 1: Collect

Runs targeted `wxcli` commands against the live org and saves results as JSON files. Includes detail collection for call queues and a permissions sample (up to 50 users).

```
# 14 collection commands + detail fetches
wxcli auto-attendant list -o json > auto_attendants.json
wxcli call-queue list -o json > call_queues.json
wxcli devices list -o json > devices.json
wxcli numbers list -o json > numbers.json
# ... plus hunt groups, paging, routing, trunks, etc.

# Detail collection per queue
wxcli call-queue show <QUEUE_ID> -o json

# Permissions sample (up to 50 users)
wxcli user-settings show-outgoing-permissions <USER_ID> -o json
```

Phase 2: Analyze

Runs `python3.14 -m wxcli.org_health.analyze` against the collected JSON. The 18 check functions are registered via a `@check()` decorator and run automatically. Output: `results.json` with findings by severity (HIGH/MEDIUM/LOW/INFO) and category scores.

Phase 3: Report

Runs `python3.14 -m wxcli.org_health.report` to generate a self-contained HTML report using the same "Authority Minimal" design system as the migration assessment report. Accepts `--brand` and `--prepared-by` flags for customization.

Check categories (18 checks)

CATEGORY	CHECKS	WHAT IT CATCHES
Security Posture	4	AA external transfers (toll fraud), queues without recording, unrestricted international dialing, no outgoing permission rules
Routing Hygiene	3	Dial plans with no route choices, orphaned route groups/lists, trunks in error state
Feature Utilization	6	Disabled AAs, understaffed queues (≤1 agent), single-member hunt groups, empty voicemail/paging groups, unused call parks
Device Health	5	Offline devices, users at 5-device limit, unassigned devices, workspaces without devices, stale activation codes

Output

REPORT FORMAT	DESIGN SYSTEM	CHECKS	CATEGORIES
Self-contained HTML	"Authority Minimal"	18 deterministic	4 (security, routing, features, devices)

> **Key design choice:** All checks are deterministic Python code, not LLM analysis. The org health module reuses the migration report's CSS design system and chart functions but has its own check registry and scoring model.

STANDALONE

Query Live Org State

On demand — "show me..." or /query-live

Ask plain-English questions about your Webex Calling environment and get answers from live API data — read-only, no changes made, no deployment plan needed.

How it works

The skill classifies your question, routes it to one or more domain modules, runs the right `wxcli` read commands, and formats the answer in admin-friendly language. Cross-domain queries load multiple modules automatically.

```
Classify question → Route to domain(s) → Run wxcli queries → Format answer
```

Domain modules (6)

YOU ASK ABOUT	DOMAIN MODULE	WHAT IT QUERIES
Users, voicemail, forwarding, DND, recording	**people-and-settings**	Person settings, call handling, permissions
Hunt groups, call queues, AAs, paging, park	**features**	Feature configs, members, schedules
Trunks, route groups, dial plans, PSTN	**routing**	Routing infrastructure, translation patterns
Phone numbers, assignment, availability	**numbers**	Number inventory and assignment status
Call history, CDR, volume, missed calls	**call-history**	Detailed call records, busiest hours
"What happens when someone calls...?"	**call-flow-trace**	Call path tracing, schedule evaluation

Example queries

```
# People + settings
"Which users at Austin don't have voicemail enabled?"
"Does John Smith have call forwarding set up?"

# Features
"Who's in the Sales hunt group?"
"Show me all auto attendants at the Denver location"

# Numbers + routing
"What number is assigned to the Main auto attendant?"
"How many unassigned phone numbers do we have?"

# Call history
"How many calls did we get yesterday?"
"What were the busiest hours last week?"

# Call flow tracing
"What happens when someone calls +1-512-555-1000?"
```

Safety guardrails

- **Read-only enforcement** — Only `list`, `show`, and `test` verbs are allowed. Create, update, and delete are blocklisted
- **Resource resolution** — References by name are resolved via list + case-insensitive match. Multiple matches trigger disambiguation
- **Batch protocol** — Audit queries (>50 users) batch in groups of 20 with progress updates
- **Handoff to builder** — If you ask to change something, the skill hands off to the build pipeline

> **When to use query-live vs. the builder:** If your question starts with "who", "what", "which", "how many", "does", "is", or "show me" — it's a query. If it starts with "create", "set up", "enable", "configure" — it's a build task. The builder agent auto-routes query-shaped requests to this skill.

RESILIENCE

Failure Modes & What the Playbook Does About Them

Reference — not a pipeline step

Builds fail. Tokens expire mid-run, extensions conflict, APIs return unexpected errors, scopes are missing. Here is what the playbook anticipates and how it recovers.

Failure mode table

WHAT COULD GO WRONG	WHAT THE PLAYBOOK DOES ABOUT IT
Token expires mid-build (401)	Stops, asks for new token, pipes via `echo | wxcli configure`, resumes from last successful step
Token missing scope (403)	Diagnoses which scope is missing by referencing `authentication.md`. Different API domains need different scopes
Resource already exists (409)	Asks you: update existing, skip, or delete and recreate. Never silently creates duplicates
Missing prerequisite (404)	Stops, reports the gap, offers to create the missing resource or skip dependent steps
Extension conflict	Caught during prerequisite check (phase 2) before any API calls are made
Context compaction mid-build	Plan is saved to disk before execution. Agent re-reads plan file and resumes from next pending step
Rate limiting (429)	wxcli auto-retries by default. For bulk loops: `sleep 1` between calls or reduce async concurrency
Partial bulk operation failure	Reports successes and failures per item. Asks: retry failed only, skip, or rollback all
API behavior differs from docs	Gotcha written back to reference docs via sync protocol. Next session benefits automatically
wxcli does not cover the operation	Falls back to raw HTTP. AA menus, license assignment, CX Essentials all use this path
Call-controls returns 400	Requires user-level OAuth token, not admin. Agent diagnoses and explains the token type requirement
Workspace settings return 405	Most `/telephony/config/workspaces/` settings require Professional license. Basic workspaces only get musicOnHold and doNotDisturb. Agent checks license tier first

Wrong location ID used silently	Agent always verifies location ID before location-scoped calls. A wrong ID silently applies settings to the wrong location

The debug skill

For errors that do not match simple patterns, `/wxc-calling-debug` provides systematic diagnosis: check token validity, check scopes, run commands with `--debug` flags, extract tracking IDs for TAC escalation, and match symptoms against a comprehensive cause/fix table covering HTTP errors, SDK behaviors, resource issues, and async-specific problems.